SKILL-7

SESSION-14

Task-1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];
    char data[MAX_INPUT];

    printf("Append Redirection Demonstration\n");
    printf("--------------------------------\n");

    /* Get append redirection input */
    printf("Enter command with append redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Parse append operator >> */
    char *redirect = strstr(input, ">>");

    if (redirect == NULL) {
        printf("Error: Append operator '>>' not found.\n");
        return 1;
    }

    /* Extract filename after >> */
    strcpy(filename, redirect + 2);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }

    /* Remove trailing spaces */
    int len = strlen(filename);
    while (len > 0 && filename[len - 1] == ' ') {
        filename[len - 1] = '\0';
        len--;
    }

    if (strlen(filename) == 0) {
        printf("Error: No output filename specified.\n");
        return 1;
    }

    printf("Parsed output file: %s\n", filename);

    /*
     * Open file in append mode.
     * O_WRONLY -> write only
```

```c
     * O_CREAT  -> create if file does not exist
     * O_APPEND -> write at the end of the file
     */
    int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    /* Save original stdout */
    int saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1) {
        perror("dup");
        close(fd);
        return 1;
    }

    /* Redirect stdout to the file */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stdout);
        return 1;
    }

    close(fd);

    printf("New data appended successfully.\n");
    printf("This line was added without deleting existing data.\n");
    printf("Append redirection using >> is working.\n");

    fflush(stdout);

    /* Restore stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Restoring stdout failed");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("stdout restored successfully.\n");
    printf("Append operation completed.\n");

    /* Verify the complete file */
    FILE *verify = fopen(filename, "r");

    if (verify == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nComplete contents of %s:\n", filename);
    printf("--------------------------------\n");

    char buffer[128];

    while (fgets(buffer, sizeof(buffer), verify) != NULL) {
        printf("%s", buffer);
    }

    fclose(verify);

    printf("--------------------------------\n");
    printf("Existing data preserved and new data appended successfully.\n");
```

```
    return 0;
}
```

```
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ nano skill7s14task1.c
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ gcc skill7s14task1.c -o skill7s14task1
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ echo "This is the original data." > append.txt
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ cat append.txt
This is the original data.
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ ./skill7s14task1
Append Redirection Demonstration
--------------------------------
Enter command with append redirection: echo New data >> append.txt
Parsed output file: append.txt
stdout restored successfully.
Append operation completed.

Complete contents of append.txt:
--------------------------------
This is the original data.
New data appended successfully.
This line was added without deleting existing data.
Append redirection using >> is working.
--------------------------------
Existing data preserved and new data appended successfully.
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ ^C
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ cat append.txt
This is the original data.
New data appended successfully.
This line was added without deleting existing data.
Append redirection using >> is working.
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ ./skill7s14task1
Append Redirection Demonstration
--------------------------------
Enter command with append redirection: echo Another append >> append.txt
Parsed output file: append.txt
stdout restored successfully.
Append operation completed.

Complete contents of append.txt:
--------------------------------
This is the original data.
New data appended successfully.
This line was added without deleting existing data.
Append redirection using >> is working.
New data appended successfully.
This line was added without deleting existing data.
Append redirection using >> is working.
--------------------------------
Existing data preserved and new data appended successfully.
```

Observation:

The program successfully parses the append redirection operator >> and extracts the output filename. The file is opened using O_WRONLY, O_CREAT, and O_APPEND, ensuring that new data is written at the end of the file without deleting existing contents. Standard output is redirected to the file using dup2(). After the append operation, the original stdout stream is restored and the complete file contents are read and displayed to verify that the existing data was preserved and the new data was appended successfully.

Task-2: To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("Standard Error Redirection Demonstration\n");
    printf("----------------------------------------\n");

    printf("Enter command with stderr redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Parse stderr redirection operator 2> */
    char *redirect = strstr(input, "2>");

    if (redirect == NULL) {
        printf("Error: stderr redirection operator '2>' not found.\n");
        return 1;
    }

    /* Extract filename after 2> */
    strcpy(filename, redirect + 2);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }

    /* Remove trailing spaces */
    int len = strlen(filename);
    while (len > 0 && filename[len - 1] == ' ') {
        filename[len - 1] = '\0';
        len--;
    }

    if (strlen(filename) == 0) {
        printf("Error: No error-output filename specified.\n");
        return 1;
    }

    printf("Parsed error output file: %s\n", filename);

    /*
     * Open/create error output file.
     *
     * O_WRONLY -> write only
     * O_CREAT  -> create if it does not exist
     * O_TRUNC  -> overwrite previous contents
     */
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("Error opening error-output file");
        return 1;
    }
```

```c
    /* Save original stderr */
    int saved_stderr = dup(STDERR_FILENO);

    if (saved_stderr == -1) {
        perror("dup");
        close(fd);
        return 1;
    }

    /* Redirect stderr to the file */
    if (dup2(fd, STDERR_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stderr);
        return 1;
    }

    close(fd);

    fprintf(stderr, "Error: Unable to open requested file.\n");
    fprintf(stderr, "Error: This message was redirected from stderr.\n");
    fprintf(stderr, "Error: Demonstration of error stream capture.\n");

    fflush(stderr);

    /* Restore original stderr */
    if (dup2(saved_stderr, STDERR_FILENO) == -1) {
        perror("Restoring stderr failed");
        close(saved_stderr);
        return 1;
    }

    close(saved_stderr);

    printf("stderr restored successfully.\n");
    printf("Error output redirection completed.\n");

    /* Verify file contents */
    FILE *verify = fopen(filename, "r");

    if (verify == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nCaptured error output from %s:\n", filename);
    printf("--------------------------------\n");

    char buffer[128];

    while (fgets(buffer, sizeof(buffer), verify) != NULL) {
        printf("%s", buffer);
    }

    fclose(verify);

    printf("--------------------------------\n");
    printf("Error output file verified successfully.\n");

    return 0;
}
```

```
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ nano skill7s14task2.c
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ gcc skill7s14task2.c -o skill7s14task2
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ ./skill7s14task2
Standard Error Redirection Demonstration
----------------------------------------
```

```
Enter command with stderr redirection: cat missing.txt 2> errors.txt
Parsed error output file: errors.txt
stderr restored successfully.
Error output redirection completed.

Captured error output from errors.txt:
--------------------------------
Error: Unable to open requested file.
Error: This message was redirected from stderr.
Error: Demonstration of error stream capture.
--------------------------------
Error output file verified successfully.
venkat@LAPTOP-LS5FC8JJ:~/Operating_Systems$ cat errors.txt
Error: Unable to open requested file.
Error: This message was redirected from stderr.
Error: Demonstration of error stream capture.
```

Observation:

The program successfully parses the stderr redirection operator 2> and extracts the specified error-output filename. The file is opened using open() and standard error is redirected to the file using dup2(). Error messages generated through stderr are captured in the specified file instead of being displayed directly on the terminal. The original stderr stream is restored after redirection, and the file is reopened to verify the captured error contents. Invalid redirection syntax and missing filenames are also handled appropriately.